SIMATS ENGINEERING
ASSESSMENT TOOL

COURSE NAME: **Theory of Computation** **COURSE CODE:** **CSA13**

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 5 **Total Marks:25**
Duration : 1 Hour

Application Based Question

Code of Conduct:

I D.Naveen / 192372177 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING
ASSESSMENT TOOL

1.In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable.Design a **Deterministic Finite Automaton (DFA)** over the alphabet Σ = {0,1} to validate passwords that **end with the pattern "01"**.Further, **minimize the DFA** and analyze how the optimized design improves system efficiency in real-world hardware implementations

2.In web applications, validating user input is essential to ensure correctness, security, and performance.Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet Σ = {a, b}.Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications**.**

3.In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role.Demonstrate how the regular expression `a*b*` can be converted into an equivalent finite automaton (FA).Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.

4.In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language L= { $a^n b^n$ | n ≥ 0 } cannot be recognized by any finite automaton.Further, analyze the implications of this limitation in the design of modern compilers.

5.Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules.Explain how the **closure properties of regular languages** enable the combination of multiple detection rules.Further, justify why the resulting language after applying these operations **remains regular**, ensuring efficient implementation using finite automata

SIMATS ENGINEERING
ASSESSMENT TOOL

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

2. Can enforce rules like:

- **Email format**
- **Password rules**
- **Input structure**

SIMATS ENGINEERING
ASSESSMENT TOOL

Theory of Computation (CSA13) | CO2 – Application Based Question | D.Naveen | Reg No: 192372177

1. DFA for passwords ending with "01" (Σ = {0,1}) and its minimization

Language: L = { w ∈ {0,1}* | w ends with 01 }.

Design idea. The machine only needs to remember how much of the suffix "01" has been matched at the end of the input read so far.

- **q0** – start; last symbol is not a useful 0 (nothing matched yet).
- **q1** – the last symbol read was 0 (the "0" of "01" is matched).
- **q2** – the last two symbols were 01 (**accepting** state).

Formal definition. M = (Q, Σ, δ, q0, F) with Q = {q0, q1, q2}, Σ = {0,1}, start state q0, F = {q2}.

State	On 0	On 1
→ q0	q1	q0
q1	q1	q2
* q2	q1	q0

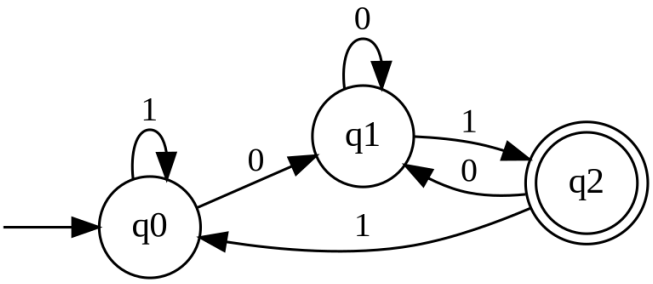


Fig. 1 – DFA for strings ending with 01 (already minimal)

Test: 1101 → q0→q0→q0→q1→q2 (accepted). 0110 → q0→q1→q2→q0→q1 (rejected). Empty string stays in q0 (rejected).

Minimization (table-filling / partition refinement).

Step 1: separate final and non-final states: {q0, q1} and {q2}.

Step 2: check q0 and q1 on every symbol. On input 1: δ(q0,1) = q0 (non-final) but δ(q1,1) = q2 (final). Hence q0 and q1 are distinguishable by the string "1".

Pair	Distinguishing string	Reason
(q0, q2)	ε	q2 is final, q0 is not
(q1, q2)	ε	q2 is final, q1 is not
(q0, q1)	1	q0·1 → q0 (reject), q1·1 → q2 (accept)

Every pair is marked, so no states can be merged. The 3-state DFA is the **minimal DFA** (it also equals the number of Myhill–Nerode classes: "ends in neither", "ends in 0", "ends in 01"). Any redundant version that duplicates states (for example one that tracks the last two bits with 4 states) reduces to exactly this machine.

How the optimized design improves real-world hardware efficiency

- **Fewer flip-flops:** 3 states need only 2 state bits (2 flip-flops); every extra redundant state in an unminimized design adds registers and next-state logic.
- **Smaller logic and area:** a 3 × 2 transition table means fewer gates/ROM entries, lower silicon area and lower cost in embedded security chips and ATM controllers.
- **Low power and high speed:** fewer switching elements gives lower dynamic power and a shorter critical path, so each input bit is processed in one clock cycle.
- **Constant memory, linear time:** one pass over the password, O(n) time, O(1) memory, no backtracking and no need to store the password.

- **Reliability and verification:** deterministic behaviour, fewer states to test and verify, and predictable timing – important for authentication modules.

2. Regular expression for the input pattern "aaaabb" (Σ = {a,b})

Regular expression: r = **aaaabb** (equivalently a^4b^2 = a·a·a·a·b·b, using concatenation).

The language is L(r) = { aaaabb }, a finite (hence regular) language containing exactly one string. Strings such as aaabb, aaaabbb, baaaab or the empty string are rejected.

For comparison, if all strings that *start with* aaaabb were wanted: aaaabb(a+b)*; strings that *contain* it: (a+b)*aaaabb(a+b)*.

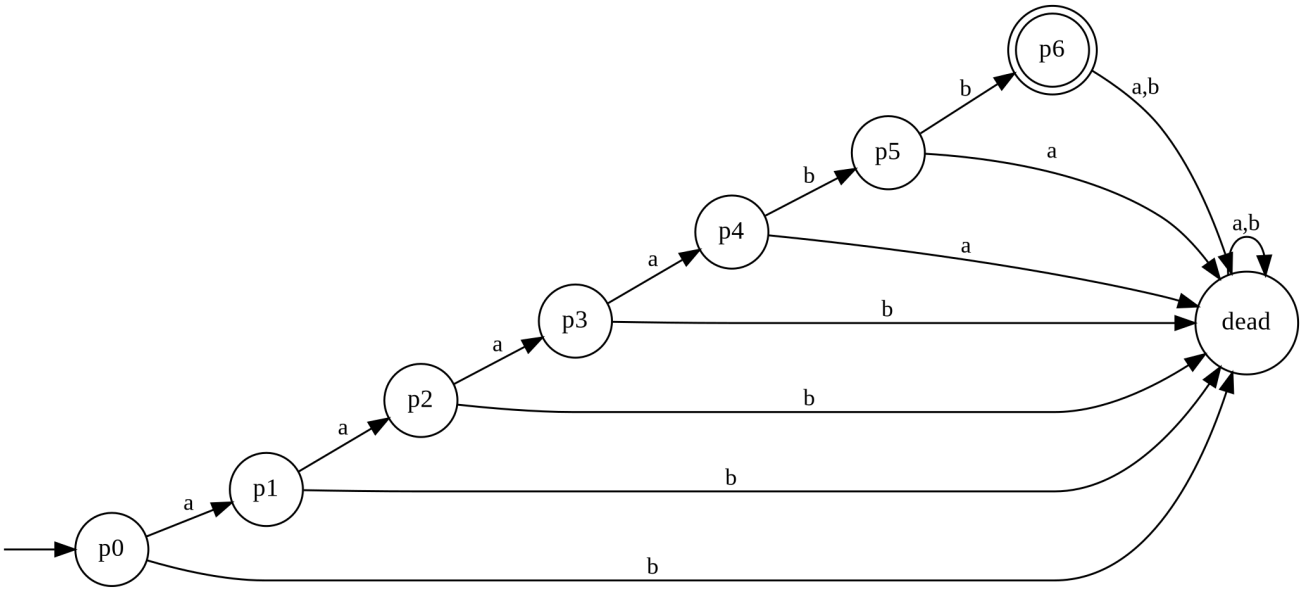


Fig. 2 – DFA for aaaabb (p6 is the only accepting state, "dead" is the trap state)

Role of regular languages and regular expressions in web applications

Every regular expression can be converted to an NFA/DFA, so a validator runs in time linear in the input length with a fixed amount of memory. This makes regex-based validation fast enough to run on every keystroke in the browser and on every request on the server.

Validation task	Typical regular expression (idea)	What it enforces
Email format	[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}	local part, one "@", domain, dot, TLD
Password rules	(?=.*[0-9])(?=.*[A-Z]).{8,}	min. length, at least a digit and an upper-case letter
Input structure	[0-9]{10} , [0-9]{6} , [A-Z]{5}[0-9]{4}[A-Z]	phone, PIN code, PAN-like fixed formats

- **Correctness:** rejects malformed input before it reaches business logic.
- **Security:** whitelisting an exact pattern blocks SQL-injection, script-injection and path-traversal characters at the input boundary.
- **Performance:** compiled automata (e.g. in JavaScript, Java, RE2) give predictable linear-time matching; client-side checks reduce server load.
- **Limitation / caution:** regular expressions cannot check nested or counted structure (e.g. balanced brackets), and badly written back-tracking patterns can cause ReDoS, so validation is combined with other checks.

3. Converting the regular expression a*b* into a finite automaton

Step 1 – Regular expression → ε-NFA (Thompson construction). a* is a loop on "a", b* is a loop on "b", and the concatenation joins the two fragments with an ε-transition. After removing the redundant Thompson helper ε-moves we get:

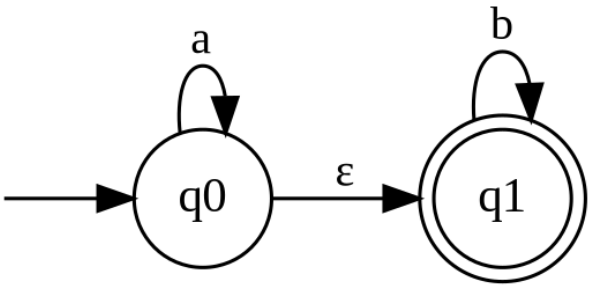


*Fig. 3 – ε-NFA for a*b**

Step 2 – Remove ε-transitions. ε-closure(q0) = {q0,q1}, ε-closure(q1) = {q1}. Since the closure of q0 contains the final state q1, q0 also becomes final.

State	On a	On b
→ * q0	{q0}	{q1}
* q1	{ }	{q1}

Step 3 – NFA → DFA (subset construction). A = {q0,q1} (start, final), B = {q1} (final), dead = { } (trap state).

DFA state	On a	On b
→ * A = {q0,q1}	A	B
* B = {q1}	dead	B
dead = { }	dead	dead

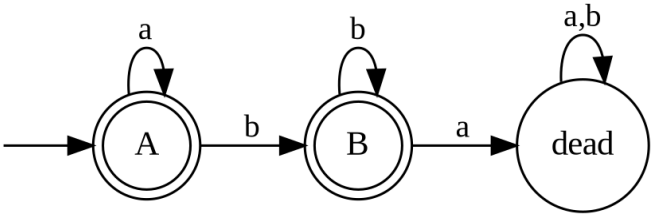


*Fig. 4 – Equivalent DFA for a*b* (all a's followed by all b's)*

The automaton accepts ε, a, b, aab, abbb, aabbb, … and rejects ba, aba, abab.

Why regular expressions and finite automata are equivalent (Kleene's theorem)

- **RE → FA:** every operator has an automaton: a single symbol, union (new start with ε to both), concatenation (ε from final of the first to start of the second) and star (ε loop and bypass). By induction on the RE, an ε-NFA exists for every RE.
- **FA → RE:** state elimination (or Arden's theorem) removes states one by one and leaves a regular expression for the same language.
- ε-NFA, NFA and DFA recognise the same class (subset construction), so REs, NFAs, DFAs and ε-NFAs all describe exactly the **regular languages**.

Use in real-time detection systems

- **Intrusion detection (e.g. Snort-style signatures):** signatures are written as REs, compiled to one DFA, and the traffic stream is scanned byte by byte in O(1) work per byte.
- **Compilers:** the lexical analyser (lex/flex) turns token REs (identifiers, numbers, keywords) into a DFA.
- **Monitoring tools / log analysis:** alert patterns are matched on streaming data with no backtracking, giving guaranteed real-time throughput, and are easy to implement in hardware (FPGA/network cards).

4. Pumping lemma: L = { $a^n b^n$ | n ≥ 0 } is not regular

Pumping Lemma. If L is regular, there is a pumping length p ≥ 1 such that every string w ∈ L with |w| ≥ p can be written as w = xyz with (1) |y| ≥ 1, (2) |xy| ≤ p, and (3) x y^i z ∈ L for all i ≥ 0.

Proof by contradiction.

- **Assume** L is regular, and let p be its pumping length.
- **Choose** w = $a^p b^p$. Then w ∈ L and |w| = 2p ≥ p.
- **Split** w = xyz. Because |xy| ≤ p, both x and y lie entirely inside the first block of a's. So y = a^k with k ≥ 1 (|y| ≥ 1).
- **Pump** with i = 2 (any i ≠ 1 works): x y^2 z = $a^{p+k} b^p$.
- Since k ≥ 1, the number of a's (p + k) is not equal to the number of b's (p), so x y^2 z ∉ L. Also i = 0 gives $a^{p-k} b^p$ ∉ L.
- This contradicts condition (3) of the pumping lemma. Hence the assumption is false and **L is not regular**, so no finite automaton recognises it.

Intuition: a DFA has only finitely many states, so it cannot count an unbounded number of a's and compare it with the number of b's. Long inputs force a state to repeat, and the loop can then be pumped up or down.

Implications in modern compiler design

- Programming languages contain nested/balanced structures – parentheses, braces { }, begin…end, nested if-else, matching HTML/XML tags – which have the same form as $a^n b^n$. A regular expression or DFA cannot check these.
- Therefore compilers use a **two-phase design**: the lexical analyser uses regular expressions/DFAs for tokens, and the syntax analyser uses **context-free grammars with pushdown automata** (LL / LR parsers) whose stack provides the counting memory.
- Even a CFG cannot check context-sensitive rules such as "a variable must be declared before use" or type-matching; these are handled in **semantic analysis** with symbol tables.
- Knowing the limits avoids wrong designs (e.g. trying to validate balanced brackets with a regex) and lets each compiler stage use the cheapest sufficient model: DFAs for speed in lexing, pushdown parsers for structure.

5. Closure properties of regular languages and combining detection rules

Let each detection rule R_i be a regular language L_i (given by a regular expression or an automaton, e.g. L_1 = strings containing "DROP TABLE", L_2 = strings containing a suspicious URL, L_3 = strings that are in a whitelist). Regular languages are **closed** under the following operations, so rules can be combined freely:

Operation	Meaning for rules	Construction on automata
Union L1 ∪ L2	alert if rule 1 OR rule 2 matches (spam signatures)	new start state with ε-moves to both NFAs, or product DFA with F = F1×Q2 ∪ Q1×F2
Intersection L1 ∩ L2	input must satisfy rule 1 AND rule 2 (password policy)	product construction: states (p,q), δ((p,q),a) = (δ1(p,a), δ2(q,a)), F = F1×F2
Complement L^c	NOT matching a rule (reject blacklisted pattern)	make the DFA complete, then swap final and non-final states
Difference L1 − L2	L1 but not L2 (rule with exceptions)	L1 ∩ complement(L2)
Concatenation L1L2	rule 1 followed by rule 2 (header then payload)	ε-move from finals of M1 to start of M2
Kleene star L*	repetition of a pattern	new start/final state with ε loop back
Reversal, homomorphism	reverse or rename symbols (encodings, case folding)	reverse arrows / replace labels

Example. Password policy: L = L_{len} ∩ L_{digit} ∩ L_{upper} ∩ complement(L_{common}), where L_{len} = length ≥ 8, L_{digit} = contains a digit, L_{upper} = contains a capital, L_{common} = contains "password". Intrusion detection: L = (L_{sig1} ∪ L_{sig2} ∪ … ∪ L_{sigk}) − $L_{trusted}$.

Why the resulting language is still regular (justification)

- **Union / concatenation / star:** they are the basic regular-expression operators (r_1+r_2, r_1r_2, r^*), so the result is again a regular expression and therefore regular.
- **Intersection:** if M_1 and M_2 are DFAs, the product automaton $M = (Q_1 \times Q_2, \Sigma, \delta, (q_{01},q_{02}), F_1 \times F_2)$ has finitely many states ($|Q_1| \cdot |Q_2|$) and by induction on $|w|$, M accepts w iff both M_1 and M_2 accept w. A finite automaton exists, so the language is regular.
- **Complement:** in a complete DFA exactly one path exists for each input; swapping accepting and non-accepting states accepts exactly the strings that were rejected, and the automaton is still finite.
- **Others** (difference, via De Morgan: $L_1 \cap L_2^c$; also $L_1 \cup L_2$ = complement of $(L_1^c \cap L_2^c)$) follow from the above.

Efficient implementation: after the operations the combined rule set is a single finite automaton; it can be determinised and minimised once (offline), after which the incoming data is scanned in one pass, one transition per symbol, with constant memory. Hence hundreds of rules in an IDS, spam filter or input validator cost the same per character as a single rule, and the machine can be built in software or hardware. Because every operation produces a finite automaton, new rules can be added, modified or negated without re-designing the system, and correctness is guaranteed by the closure theorems.